- EmotionHub 指南
  - 项目简介
  - 快速启动
    - 1. 启动数据库
    - 2. 启动后端
    - 3. 启动前端
  - 测试账号
  - 核心功能演示
    - 1. 用户注册和登录
    - 2. 发布帖子
    - 3. 情感分析
    - 4. 互动功能
    - 5. 通知中心
    - 6. 个人中心
  - 高级功能
    - 1. 接口限流
    - 2. 敏感词过滤
    - 3. Redis缓存
  - API文档
    - 认证接口
    - 帖子接口
    - 互动接口
    - 通知接口
    - 统计接口
  - 配置说明
    - 数据库配置
    - Redis配置
    - 通义千问配置
    - JWT配置
    - 线程池配置
  - 项目结构
  - 前端页面
  - 许可证

EmotionHub 指南

项目简介

EmotionHub 是一个基于情感分析的社交媒体平台，具有以下特色：

- **真实LLM集成**：支持通义千问API进行智能情感分析
- **完整的社交功能**：发帖、评论、点赞、通知
- **Archive Vault设计**：独特的1940s档案馆美学
- **企业级架构**：Redis缓存、限流、敏感词过滤

快速启动

1. 启动数据库

```
# 在项目根目录执行
docker-compose up -d
```

这将启动：

- MySQL 8.0（端口3306）
- Redis（端口6379）

2. 启动后端

方式一：使用Maven（推荐）

```
cd backend/emotion-hub-web
mvn spring-boot:run
```

方式二：IDE运行

- 在IDE中打开 `EmotionHubWebApplication.java`
- 点击运行按钮

后端启动后访问：

- API地址：http://localhost:8080/api
- API文档：http://localhost:8080/doc.html

3. 启动前端

```
cd frontend
npm install  # 首次运行
npm run dev
```

前端访问：http://localhost:3001

测试账号

所有测试账号密码都是：`password123`

用户名	昵称	邮箱
alice_chen	Alice Chen	alice@example.com
bob_wang	Bob Wang	bob@example.com
carol_liu	Carol Liu	carol@example.com
david_zhang	David Zhang	david@example.com
emma_li	Emma Li	emma@example.com

核心功能演示

1. 用户注册和登录

注册流程：

1. 访问 http://localhost:3001/login
2. 点击"REGISTER"标签
3. 填写用户名、昵称、邮箱、密码
4. 点击"CREATE ACCOUNT"
5. 自动登录并跳转到广场

登录流程：

1. 访问 http://localhost:3001/login
2. 输入用户名和密码
3. 点击"ACCESS ARCHIVE"
4. 登录成功，跳转到广场

2. 发布帖子

1. 登录后，点击右上角"+ NEW ENTRY"
2. 输入帖子内容
3. （可选）上传图片（最多9张）
4. 点击"SUBMIT ENTRY"
5. 帖子发布后，后台会**异步进行情感分析**

3. 情感分析

关键词分析（默认）：

- 系统会自动分析帖子中的正面/负面关键词
- 计算情感分数（-1.0到1.0）
- 标注情感标签（POSITIVE/NEGATIVE/NEUTRAL）

通义千问AI分析（可选）：

1. 在 `application-dev.yml` 中配置：

```
dashscope:
  api-key: sk-xxxxxxxxxxxxx  # 你的通义千问API Key

emotion:
  analysis:
    provider: qwen  # 改为qwen
```

2. 重启后端，发布帖子会使用通义千问AI分析
3. 如果API失败，会自动降级到关键词分析

4. 互动功能

点赞：

- 点击帖子卡片进入详情页
- 点击"♡ LIKES"按钮进行点赞
- 再次点击取消点赞（幂等操作）
- 帖子作者会收到点赞通知

评论：

- 在帖子详情页底部的"CASE NOTES"区域
- 输入评论内容
- 点击"SUBMIT NOTE"
- 支持回复评论（树形结构）
- 帖子作者/被回复者会收到通知

5. 通知中心

1. 点击右上角"MESSAGES"（有红色数字徽章表示未读）
2. 查看未读/全部通知
3. 点击通知可跳转到相关帖子
4. 点击"MARK ALL READ"标记全部已读

6. 个人中心

1. 点击右上角"PROFILE"
2. 查看个人统计：
    - 总帖子数
    - 总点赞数
    - 总评论数
3. 查看情感分析图表：
    - 正面情感百分比
    - 负面情感百分比
    - 中性情感百分比
4. 查看个人发布的所有帖子

高级功能

1. 接口限流

使用方法：

在Controller方法上添加注解：

```
@RateLimit(time = 60, count = 10, limitType = RateLimit.LimitType.USER)
@PostMapping("/post")
public Result<PostVO> createPost(@RequestBody PostCreateRequest request) {
    // ...
}
```

参数说明：

- `time`: 时间窗口（秒）
- `count`: 时间窗口内最大请求数
- `limitType`:
  - `DEFAULT`: 按IP限流
  - `USER`: 按用户ID限流
  - `GLOBAL`: 全局限流

效果：

- 超过限流阈值会返回：`{"code": 6001, "message": "操作过于频繁，请稍后重试"}`

2. 敏感词过滤

使用方法：

```
@Autowired
private SensitiveWordService sensitiveWordService;

// 检查是否包含敏感词
if (sensitiveWordService.contains(content)) {
    throw new BusinessException(ErrorCode.SENSITIVE_WORD_DETECTED);
}

// 过滤敏感词（替换为***）
String filtered = sensitiveWordService.filter(content);
```

```
// 获取所有敏感词
Set<String> words = sensitiveWordService.getSensitiveWords(content);
```

敏感词库：

- 位于 `SensitiveWordServiceImpl.loadSensitiveWords()`
- 基于DFA（确定有穷自动机）算法，高效匹配
- 支持最小/最大匹配模式

3. Redis缓存

使用方法：

```
@Autowired
private CacheService cacheService;

// 设置缓存
cacheService.set(
    CacheService.CacheKey.POST_DETAIL + postId,
    post,
    CacheService.CacheTTL.POST_DETAIL,
    TimeUnit.SECONDS
);

// 获取缓存
Post post = cacheService.get(
    CacheService.CacheKey.POST_DETAIL + postId,
    Post.class
);

// 删除缓存
cacheService.delete(CacheService.CacheKey.POST_DETAIL + postId);
```

预定义缓存Key：

- `post:detail:` - 帖子详情（5分钟）
- `post:hot` - 热门帖子（10分钟）
- `user:info:` - 用户信息（30分钟）
- `stats:user:` - 用户统计（1小时）
- `emotion:trend:` - 情感趋势（2小时）

API文档

访问 http://localhost:8080/doc.html 查看完整的API文档

主要接口：

认证接口

- `POST /api/auth/register` - 用户注册
- `POST /api/auth/login` - 用户登录
- `GET /api/user/info` - 获取当前用户信息

帖子接口

- `POST /api/post` - 发布帖子
- `GET /api/post/list` - 查询帖子列表
- `GET /api/post/{id}` - 获取帖子详情
- `DELETE /api/post/{id}` - 删除帖子

互动接口

- `POST /api/interaction/like` - 点赞/取消点赞
- `POST /api/interaction/comment` - 发表评论
- `GET /api/interaction/comment/list` - 获取评论列表
- `DELETE /api/interaction/comment/{id}` - 删除评论

通知接口

- `GET /api/notification/unread` - 获取未读通知
- `GET /api/notification/all` - 获取所有通知
- `PUT /api/notification/{id}/read` - 标记已读
- `PUT /api/notification/read-all` - 全部标记已读
- `GET /api/notification/unread-count` - 未读数量

统计接口

- `GET /api/stats/user/{userId}` - 获取用户统计

- `GET /api/stats/overview` - 系统概览

配置说明

数据库配置

`application-dev.yml`:

```yaml
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/emotion_hub
    username: root
    password: emotion_hub_2024
```

Redis配置

```yaml
spring:
  data:
    redis:
      host: localhost
      port: 6379
      password:  # 默认无密码
      database: 0
```

通义千问配置

```yaml
dashscope:
  api-key: ${DASHSCOPE_API_KEY:}  # 环境变量或直接填写

emotion:
  analysis:
    provider: keyword  # keyword或qwen
```

JWT配置

```yaml
jwt:
  secret: EmotionHub_Secret_Key_2024_Change_This_In_Production
  expiration: 604800000  # 7天
```

线程池配置

```yaml
async:
  executor:
    core-pool-size: 5
    max-pool-size: 20
    queue-capacity: 100
```

项目结构

```
EmotionHub/
├── backend/                    # 后端项目
│   ├── emotion-hub-common/     # 公共模块
│   │   ├── enums/              # 枚举类
│   │   ├── exception/          # 异常类
│   │   ├── annotation/         # 注解（@RateLimit）
│   │   └── result/             # 统一返回结果
│   ├── emotion-hub-model/      # 数据模型
│   │   ├── entity/             # 实体类
│   │   ├── dto/                # 数据传输对象
│   │   └── vo/                 # 视图对象
│   ├── emotion-hub-dao/        # 数据访问层
│   │   └── mapper/             # MyBatis Mapper
│   ├── emotion-hub-service/    # 业务逻辑层
│   │   ├── impl/               # Service实现
│   │   └── cache/              # 缓存服务
│   └── emotion-hub-web/        # Web层
│       ├── controller/         # REST API
│       ├── config/             # 配置类
│       ├── filter/             # 过滤器
│       └── aspect/             # AOP切面（限流）
├── frontend/                   # 前端项目
│   ├── src/
│   │   ├── api/                # API封装
│   │   ├── components/         # 组件
│   │   ├── views/              # 页面
│   │   ├── stores/             # Pinia状态
│   │   ├── styles/             # Archive Vault主题
│   │   └── router/             # 路由
│   └── public/
├── docs/                       # 文档
```

```
│   ├── 快速使用指南.md
│   ├── 功能实现清单.md
│   └── 测试账号.md
└── docker-compose.yml          # Docker配置
```

前端页面

- **登录/注册页**：`/login` - Archive Vault风格的认证界面
- **帖子广场**：`/` - Manila文件夹风格的帖子卡片
- **帖子详情**：`/post/:id` - 完整帖子内容和树形评论
- **个人中心**：`/profile` - 用户统计和情感分析图表
- **通知中心**：`/notifications` - 消息通知列表

许可证

MIT License

最后更新： 2026-03-10 **项目版本：** 1.0.0 **作者：** EmotionHub Team